CTIS411 Senior Project

2025-2026

Software Requirements Specification

SRS

An Eye

Team 12

Faik Emre Partal, 22203786

Halil İbrahim Atalay, 22103092

Mevlüt Yılmaz, 22201947

Semih Şengül, 22202588

Uğur Ayberk Karakış, 22201725

Bilkent University

Department of Information Systems and Technologies

10.11.2025

Change History

File Name	Document Type	Deliverable	Version	Submission Date
Team12_SRS_V1	MS Word 2013	2A	1	10.11.2025

Project Details

Project Name	**An Eye**
Software Product Name	**ObservAI**
Company Name	**No sponsor company**
Academic Advisor	**Leyla Sezer**
Github URL	**https://github.com/observaianeye/ObservAI**
WEB page	
Software Product Information	ObservAI is an AI-powered analytics system designed to help café and restaurant managers better understand their customers and optimize their spaces. Using camera-based analysis, the system observes how people move within the area, identifies crowded zones, estimates basic demographics such as age and gender ranges, and visualizes customer activity through live heatmaps. These insights are presented on an intuitive web dashboard where managers can monitor customer flow, compare different time periods, and make data-informed decisions about layout or service improvements. The system's purpose is to transform everyday visual data into clear and actionable information that supports smarter management and smoother operations. ObservAI also prioritizes user privacy and ethical use of AI by processing all data anonymously and without storing personal or identifying visuals.

Team 12

Name, Surname	Bilkent e-mail	Alternative e-mail
Faik Emre Partal	emre.partal@ug.bilkent.edu.tr	femrepartal@gmail.com
Halil İbrahim Atalay	ibrahim.atalay@ug.bilkent.edu.tr	atalayhalil155@gmail.com
Mevlüt Yılmaz	mevlut.yilmaz@ug.bilkent.edu.tr	yilmazmevlut397@gmail.com
Semih Şengül	semih.sengul@ug.bilkent.edu.tr	semihyahama@gmail.com
Uğur Ayberk Karakış	ayberk.karakis@ug.bilkent.edu.tr	ayberkarakis@hotmail.com

Individual Contributions Overview

Name, Surname	Summary of Contributions to the SRS Document
Faik Emre Partal	Main responsible of the Document Formatting Contributed to Executive Summary, Functional Requirements and Verification Main responsible of all Gen AI usage
Halil İbrahim Atalay	Prepared the non-functional requirements and user interface sections. Assisted in defining product scope and technical descriptions.
Mevlüt Yılmaz	Worked on software interfaces and system architecture descriptions. Contributed to camera integration and AI module explanation.
Semih Şengül	Prepared the user characteristics and product perspective sections. Assisted with diagrams and figures in the SRS document.
Uğur Ayberk Karakış	Contributed to database requirements and assumptions/dependencies. Supported proofreading and consistency checks across all sections.

Executive Summary

This Software Requirements Specification (SRS) document defines the scope, objectives, and detailed requirements of ObservAI, an AI-powered analytics platform developed by Team 12 to help café and restaurant managers make data-driven decisions using real-time computer vision insights. The document provides a complete description of the software system, including its functional and non-functional requirements, system interfaces, database design, and verification approach. ObservAI is designed as a web-based platform that integrates camera-based analytics, artificial intelligence, and interactive data visualization within a single management dashboard. Through live video streams, the system detects customers, estimates demographics, and visualizes spatial heatmaps to assist managers in understanding crowd behavior and optimizing their operations. The SRS also explains the characteristics of the sole user group managers highlighting the system's focus on usability, accessibility, and ease of interpretation for non-technical users. Each major component of the system, including the frontend (React), backend (Node.js), and AI modules (YOLOv8 and OpenCV), is documented to show how they interact securely through HTTPS and comply with KVKK and GDPR regulations for anonymized data handling. In addition to defining the product scope, the SRS elaborates on software quality attributes such as reliability, maintainability, and security, supported by logical database requirements and assumptions. The preparation of this document involved close team collaboration, research on software engineering standards, and iterative refinement based on instructor feedback. Throughout the preparation process, the team encountered several challenges such as managing academic workload alongside documentation tasks, maintaining consistency across sections, and adapting the report to formal IEEE SRS standards. Despite these difficulties, regular meetings and peer reviews helped ensure that the document remained clear, coherent, and aligned with project objectives. Despite these obstacles, consistent teamwork, weekly discussions, and the use of shared development tools helped ensure coherence and quality. As a result, this document serves as a structured and verifiable foundation for the future design and implementation of ObservAI. It ensures that all requirements are traceable, realistic, and measurable, providing a clear roadmap for building a reliable and ethically compliant AI-based café management solution.[1]

[1] ChatGPT5 Prompt: **"Can you write an executive summary for this document? It should be exactly one page."**

Table of Contents

Page Number

Executive Summary iv

Table of Contents v

List of Tables viii

List of Figures ix

Abbreviations x

1. Product Scope 1

2. User Characteristics 3

3. Product Perspective 5

3.1. System Interfaces 5

3.1.1. Camera Stream Interface 5

3.1.2. Analytics Engine Interface 5

3.1.3. Database Interface 6

3.1.4. Web Server Interface 6

3.2. User Interfaces 6

3.2.0. Landing Page: 6

3.2.1. Login Page 6

3.2.2. Dashboard Home 6

3.2.3. Camera/Layering Page 6

3.2.4. AI Suggestion 6

3.2.5. Settings Page 6

3.2.6. Help Page: 7
3.2.7. Notification Page: 7
3.3. Hardware Interfaces 7
3.4. Software Interfaces 8
3.5. Communications Interfaces 10
3.6. Memory Constraints 10
4. Assumptions and Dependencies 11
5. Functional Requirements 13
5.1. List of Functional Requirements in Textual Format 13
5.2. Use Case Diagram(s) 15
5.3. Mapping of Functional Requirements 16
6. Non-Functional Requirements 17
6.1 Usability Requirements 17
6.2 Performance Requirements 18
6.3 Software System Attributes 18
6.4 Constraints 19
6.5 Error-Handling Requirements 19
6.6 Other Non-Functional Requirements 20
7. Logical Database Requirements 21
8. Verification 23
8.1. Verification Strategy 23
8.2. Functional Requirements Verification 23
8.3. Non-Functional Requirements Verification 25
8.4. Tools and Continuous Verification 26
9. Discussions 27
1. Limitations and Constraints 27

2. Health and Safety Issues 27

3. Legal Issues 28

4. Economic Issues and Constraints 28

5. Sustainability 28

6. Ethical Issues 28

7. Multidisciplinary Collaboration 29

8. Additional General Challenges 29

10. References 30

List of Tables

Page Number

Table 1 Cross Referencing Functional Requirements to Use Cases 16

Table 2 Test Tools & Purposes 26

List of Figures

Page Number

Figure 1 Use Case Diagram 15

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CI	Continuous Integration
CSS	Cascading Style Sheets
CTIS	Information Systems and Technologies
CPU	Central Processing Unit
DBMS	Database Management System
FREQ	Functional Requirement
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit
GTX	Giga Texel Shader eXtreme
HTTPS	Hypertext Transfer Protocol Secure
IEEE	Institute of Engineers Everyone Else
IDE	Integrated Development Environment
IP	Internet Protocol
JSON	JavaScript Object Notation
KVKK	Kişisel Verilerin Korunma Kanunu
LLM	Large Language Model
NFREQ	Non-Functional Requirement
OWASP	Open Web Application Security Project
RTSP	Real-Time Streaming Protocol
SRS	Software Requirements Specification
SSL	Secure Sockets Layer
SPMP	Software Project Management Plan
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modeling Language
YOLO	You Only Look Once (Object Detection Model)
WCAG	Web Content Accessibility Guidelines
WBS	Work Breakdown Structure
ZAP	Zed Attack Proxy

1. Product Scope

The software product under development is named ObservAI, a web-based artificial intelligence and computer vision platform designed for real-time customer analytics in cafés and restaurants. The system aims to provide managers with live visual insights into customer behavior, occupancy trends, and spatial usage through camera-based analysis supported by AI-driven interpretation.

ObservAI operates as a centralized, browser-accessible analytics environment that integrates computer vision capabilities with intelligent data visualization. It continuously processes live camera feeds to detect and count individuals, estimate basic demographics such as approximate age groups or gender distribution, and generate real-time heatmaps to indicate customer movement density and area utilization. These analytics are presented through a unified web interface, allowing managers to monitor and interpret on-site activity in an intuitive and data-informed way.

The application consists of a single administrative panel which serves as the primary point of access for authorized managerial users. This dashboard consolidates all analytics, AI insights, and visualization components. No additional user roles or public access levels are provided, as the application is designed exclusively for management use. The admin panel enables the viewing of live camera analyses, historical trends, and AI-generated suggestions regarding crowd flow, customer distribution, and operational optimization.

ObservAI does not include mobile or desktop native applications; however, it is fully responsive and compatible across devices such as tablets, laptops, and smartphones through standard web browsers. The design ensures that the platform maintains consistent usability and readability regardless of screen size.

From a technical standpoint, the platform's front-end is developed using React, ensuring a dynamic and responsive interface, while the back end is powered by Node.js, managing data flow, model execution, and API endpoints. The AI modules—implemented using open-source computer vision frameworks—run inference on real-time video streams to produce continuous

analytical output without manual intervention. A visualization layer transforms this raw data into interpretable formats, such as charts, demographic summaries, and spatial heatmaps.[2]

The benefits of the ObservAI system are rooted in its ability to offer actionable intelligence directly from visual data without the need for manual data collection. It helps café and restaurant managers understand customer dynamics, optimize space usage, and predict peak activity periods through AI-assisted suggestions. In doing so, it provides a foundation for data-driven operational decision-making and improves awareness of real-time in-store patterns.

The product's scope, consistent with the Initial Plan and Project Scope, deliberately excludes non-analytical features such as staff management, point-of-sale integration, and inventory tracking. ObservAI is not a complete business management system; instead, it focuses solely on transforming camera-based data into meaningful business intelligence.

[2] ChatGPT 5 Prompt: **"Brief description of the product scope focusing on technical details one web-based manager panel accessible from all devices, providing real-time AI-generated insights and suggestions, consistent with the initial project scope."**

2. User Characteristics

The ObservAI system is designed primarily for a single user group: café or restaurant managers. Since the software's main purpose is to provide real-time analytics and AI-driven insights based on camera data, it does not target general employees, customers, or technical operators. Instead, its usability, interface, and access permissions are all shaped around managerial needs and capabilities.

U1: Manager Users

- **Educational Level:**

  Manager users typically possess at least a high school education, with many holding vocational or university-level degrees in fields such as business management, hospitality, or restaurant operations. They are not expected to have formal technical education in artificial intelligence, computer science, or software engineering.

- **Demographics:**

  The primary users of the system are café and restaurant managers, generally between 25 and 55 years of age. They are professionals responsible for supervising staff, monitoring customer activity, and making operational decisions. Their work environments are usually small to medium-sized hospitality businesses.

- **Experience:**

  Managers are experienced in handling day-to-day operations, evaluating customer behavior, and ensuring service quality. They are accustomed to using business tools such as scheduling dashboards, point-of-sale systems, and reporting applications. Their interaction with ObservAI is driven by the need to quickly interpret AI-based analytics and apply them to decision-making, rather than by technical curiosity or system administration.

- **Disabilities and Accessibility:**

ObservAI is primarily designed for indoor managerial use, focusing on comfort and readability rather than full accessibility compliance. The interface provides high color contrast, legible text, and adjustable visual components to support users with mild visual impairments. However, the system does not target public or multi-role accessibility standards, as it is intended exclusively for managerial users.

- **Technical Expertise:**

Most manager users have a basic to moderate level of digital literacy. They are comfortable using web browsers, emails, and analytics dashboards but may not have technical skills related to configuration, coding, or data systems. ObservAI's design, therefore, emphasizes intuitive interaction, minimal setup requirements, and clear visual feedback, ensuring ease of use without technical training.

- **Devices and Environment:**

Managers are expected to access ObservAI from standard computing devices, including desktop computers, laptops, or tablets. The system is primarily used in office spaces or back-office areas within cafés or restaurants. ObservAI's interface is fully responsive, maintaining consistent usability and visual clarity across different screen sizes and devices.[3]

[3] ChatGPT 5 Prompt: "**description: now for the user characteristics part (2. User Characteristics) Specify the different types of users that your software product is going to have, specify their distinctive characteristics. Categorize the intended users of our software product in groups and describe those general characteristics of the intended groups of users, including characteristics that may influence usability, such as educational level, experience, disabilities, and technical expertise.**

do: write the necessary text based on our product scope and functional requirements"

3. Product Perspective

ObservAI is designed as a standalone, full-stack web application that operates independently on its own server environment. The system integrates its front-end (React-based web interface) and back-end (Node.js + AI inference layer) within a unified architecture. While it functions autonomously, the backend exposes RESTful APIs, allowing potential future integrations with larger restaurant ecosystems or third-party dashboards.

For the current academic implementation, ObservAI remains self-contained, performing all core analytics, data processing, and AI computations internally. However, its modular structure ensures easy adaptability for broader enterprise environments, where camera analytics and AI modules could be integrated into existing management software through secure API endpoints.

Hosting for ObservAI will primarily depend on cloud services such as AWS, Render, or Vercel, ensuring scalability and remote access. However, due to budget and resource constraints, local hosting alternatives may be used during development or testing phases.

3.1.System Interfaces

ObservAI's primary system interfaces are designed to manage real-time video input, AI analytics, and user interaction efficiently.

3.1.1. Camera Stream Interface

The system connects to IP cameras using RTSP protocols, fetching live video data for real-time analytics. For testing purposes, this interface can also load pre-recorded videos.

3.1.2. Analytics Engine Interface

The backend's AI module processes live video frames, detects individuals, and produces anonymized metrics such as crowd density, dwell time, and movement heatmaps. The AI layer communicates with the main backend via RESTful API calls to update the database and dashboard in real time.

3.1.3. Database Interface

ObservAI uses PostgreSQL with TimescaleDB to store time-series data (e.g., customer count over time, movement trends). Local caching and short-term storage are handled using SQLite on the edge device.

3.1.4. Web Server Interface

All data is accessed and transmitted via secure HTTPS connections. The frontend communicates with the backend through standardized JSON-based APIs for fast, secure data exchange.

3.2. User Interfaces

3.2.0. Landing Page:

Serves as the entry screen with the ObservAI logo, brief intro, and a “Login” button leading to the authentication page.

3.2.1. Login Page

Provides secure access through username and password authentication. User credentials are verified via the backend API, and all sessions are encrypted via HTTPS.

3.2.2. Dashboard Home

Displays live camera analytics, including real-time crowd detection, demographic estimation, and heatmap visualizations.

3.2.3. Camera/Layering Page

Allows users to select and view live camera feeds individually or in layered mode. Users can toggle visual overlays such as heatmaps, detection boxes, or region zones for better analysis.

3.2.4. AI Suggestion

Presents AI-generated observations and recommendations such as peak hours, customer density patterns, and suggested operational adjustments.

3.2.5. Settings Page

Enables camera configuration, threshold adjustments, and basic system preferences.

3.2.6. Help Page:

Provides user guidance and troubleshooting information. Includes step-by-step explanations for system features, contact support details, and frequently asked questions.

3.2.7. Notification Page:

Displays system alerts and status messages such as connectivity issues, detection updates, or important system warnings in real time.

The interface design follows Material UI and Tailwind CSS principles for a clean, responsive layout across all devices. Pages maintain consistent visual themes with clear typography, color-coded visuals, and tooltip-based assistance to enhance readability and reduce cognitive load.

Do's

- Maintain consistency in layout and color schemes.
- Ensure charts, maps, and video overlays are clearly labeled.
- Provide accessible font sizes and adjustable visualization density.
- Keep dashboards minimalistic and focused on essential insights.

Don'ts

- Avoid overcrowding analytical views with excessive metrics.
- Refrain from including unnecessary navigation steps.
- Avoid overly technical terminology in AI explanations.

3.3.Hardware Interfaces

ObservAI connects primarily with IP cameras using RTSP protocols to fetch real-time video streams. Each camera's stream is processed by the backend AI system, which performs frame extraction and inference on GPU-enabled environments when available.

For simulations, the system supports local file inputs (MP4 format), ensuring flexibility during testing. The backend operates on standard x86-based servers or cloud instances with moderate GPU acceleration (e.g., NVIDIA T4 or GTX 1660).

No direct physical sensors or external hardware devices are used aside from cameras and the hosting server.[4]

3.4.Software Interfaces

The following software components form the backbone of ObservAI's functionality:

Database Management System

- Name: PostgreSQL + TimescaleDB
- Mnemonic: DBMS
- Version: Latest stable
- Source: PostgreSQL Global Development Group
- Purpose: Stores time-series analytics data (e.g., customer counts, dwell durations, movement heatmaps).

Local Cache Database

- Name: SQLite
- Mnemonic: CacheDB
- Purpose: Temporary caching of results for real-time inference on local or edge devices.

AI Libraries

- Name: YOLOv8-nano, OpenCV, PyTorch
- Mnemonic: AI Modules
- Version: Latest stable releases
- Source: Ultralytics, OpenCV Foundation, PyTorch.org
- Purpose: Real-time object detection, person tracking, and video frame analysis.

Frontend Framework

- Name: React + Material UI + Tailwind CSS

[4]ChatGPT 5 Prompt: **"These are the answers of the questions: ObservAI is a standalone web app (React frontend + Node.js backend with AI layer) hosted on AWS/Vercel or locally. It connects to real IP cameras via RTSP, built with YOLOv8, OpenCV, and PyTorch. Uses PostgreSQL + TimescaleDB and SQLite for caching. Multi-page UI styled with Material UI/Tailwind, accessible via HTTPS, no strict memory limits. According those answers give the necessary text"**

- Purpose: Develops the interactive, responsive manager dashboard and visualizes data with minimal latency.

Backend Framework

- Name: Node.js + Express
- Purpose: API management, AI integration, and secure data exchange between AI layer and frontend.

Operating Systems

- Name: Windows, macOS
- Mnemonic: OS
- Version number: Windows 10 or higher / macOS Ventura or higher
- Source: Microsoft Corporation, Apple Inc.

The ObservAI platform shall function identically across the latest two versions of Chrome, Edge, Safari, and Firefox. It shall be deployable on Windows and macOS servers without code modification. All services shall run inside Docker containers to ensure consistent behavior across environments. Migration to cloud platforms such as AWS ECS or Azure Container Apps shall require minimal configuration.

Programming Environment

- Name: Visual Studio Code, PyCharm
- Mnemonic: IDE
- Version number: Latest stable versions
- Source: Microsoft, JetBrains

Visual Studio Code and PyCharm serve as the integrated development environments for building and maintaining ObservAI's backend and AI modules. These environments support debugging, testing, and continuous integration workflows, ensuring efficient and modular development of both the AI pipeline and the web application.

Notification System Specifications

- Name: Firebase Cloud Messaging (FCM), Web Push API
- Mnemonic: Notification System
- Version number: Latest stable versions
- Source: Google

The notification system enables real-time alerts and system updates for managerial users. Notifications may include alerts about unusual customer patterns, high-traffic warnings, or camera connectivity issues. Using FCM and the Web Push API ensures reliable, cross-platform message delivery for both desktop and mobile web environments.

Large Language Model (LLM) Integration

- Name: Gemini 2.0 Flash
- Mnemonic: LLM
- Version number: Latest stable release
- Source: Google DeepMind

  The LLM module integrates Gemini 2.0 Flash to provide advanced natural language processing and AI-driven insight generation within ObservAI. It converts analytical outputs and visual data into human-readable summaries, explanations, and operational recommendations. The model operates through API calls, ensuring seamless interaction with ObservAI's backend and enabling scalable, real-time generation of contextual insights for managerial users.

3.5.Communications Interfaces

ObservAI communicates exclusively through internet-based HTTPS connections, ensuring secure transmission of analytics data and video metadata.

- **Protocol**: HTTPS (Port 443)
- **Encryption**: SSL/TLS
- **Data Format**: JSON
- **Network Environment**: Cloud-hosted or local LAN setups during testing This setup ensures encrypted, real-time data flow between the backend, frontend, and connected camera streams.

3.6.Memory Constraints

ObservAI is a web-based system that runs on contemporary hosting environments. The local server configuration or cloud infrastructure takes care of memory management and scaling automatically. No specific hardware-level memory constraints apply beyond ensuring adequate system resources for real-time video inference.

4. Assumptions and Dependencies

Email System and User Authentication Services

Example: The system assumes that a reliable email service is available for verification and password recovery during user registration. If the email service provider changes, or if external email verification is restricted by organizational policies, the account creation and verification process may be affected.

This would impact FReq1.

Network and Internet Connectivity

Example: ObservAI depends on stable internet connectivity for real-time camera streaming and dashboard updates. Any network latency, outage, or restricted Wi-Fi conditions could delay live analytics, disrupt user access, or cause incomplete visual data displays.

This would affect FReq2 and FReq3.

Camera and Video Stream Availability

Example: The system assumes the availability and proper configuration of IP cameras providing live RTSP streams. If a camera becomes disconnected, misconfigured, or uses an incompatible format, real-time customer analytics cannot be performed.

This would impact FReq2 and FReq4.

AI Model and Library Dependencies

Example: ObservAI's analytics rely on open-source computer vision libraries such as YOLOv8, OpenCV, and PyTorch. Any changes, deprecations, or compatibility issues in these frameworks could affect detection accuracy, performance, or inference speed.

This would influence FReq2 and FReq3.

Hardware and Processing Resources

Example: The system assumes adequate processing capability, including access to GPU-enabled hardware or cloud resources for real-time inference. If only CPU-based hardware is

available, analytics and heatmap generation might experience significant delays. This would impact FReq2and FReq3.

Browser and Device Compatibility

Example: ObservAI is designed to function across modern web browsers and devices. If users access the system from outdated browsers or devices that do not support essential web technologies (e.g., WebGL modern CSS standards), visualization and dashboard responsiveness may degrade.

This would affect FReq3.

Data Privacy and Regulatory Compliance

Example: The system assumes that camera analytics comply with local and international privacy regulations (e.g., GDPR) and that captured video data are processed in anonymized form. If privacy laws change or stricter anonymization is required, AI detection and visualization functions may need revision.

This would impact FReq2 and FReq4.

Hosting and Infrastructure Environment

Example: ObservAI assumes deployment on a reliable hosting environment (e.g., AWS, Render, or local on-premise server). Any change in hosting provider, server configuration, or database environment may affect data availability, latency, or notification delivery.

This would influence FReq2 and FReq4.[5]

[5] ChatGPT 5 Prompt: **"Write the Assumptions and Dependencies for the ObservAI system according to the previously defined Product Scope and Product Perspective. List the factors that affect the requirements in the SRS such as the email verification system, camera interfaces, hardware resources, software libraries, hosting environment, user behavior, and regulatory compliance while noting that these are not design constraints but variables whose changes may require updates to the SRS."**

5. Functional Requirements

This section specifies the functional behavior and interaction design of the ObservAI system. It defines what the software shall do to meet user and business objectives, how users and external components interact with it, and how these behaviors are represented and traced. Subsection 5.1 details the Functional Requirements, Subsection 5.2 presents the corresponding UML Use Case Diagram, and Subsection 5.3 provides a cross-reference table linking the functional requirements, use cases, and the high-level requirements defined in the Initial Plan.[6]

5.1.List of Functional Requirements in Textual Format

FReq 1: Operations Dashboard

Description:

The system shall provide an AI-powered operations dashboard that presents real-time information on customer activity and in-store behavior.

Inputs / Stimuli:

- Continuous video streams from installed camera devices.
- Time and zone metadata associated with each feed.
- Manager login credentials (email address and password).

Process / Function:

- The computer-vision module analyzes frames to detect and count visitors.
- The authentication module validates that the manager's email address and password correspond to a registered account.
- Upon successful authentication, the manager gains access to the dashboard interface.
- The system estimates age range and gender distribution.
- Detected data are aggregated and sent to the dashboard service.

Outputs / Responses:

- Real-time visual elements (heatmaps, charts, numeric widgets) displayed to the manager on the dashboard.
- Login success or failure messages displayed based on authentication results.

[6] ChatGPT 5 Prompt: **"Write an opening part for this section."**

FReq 2: Interactive Visualization and Reporting

Description:

The system shall allow the manager to interactively switch between visualization modes and to view AI-generated analytics without leaving the dashboard.

Inputs / Stimuli:

- User click or touch action selecting a chart type (bar, line, heatmap) or report view.

Process / Function:

- The visualization engine renders the same data in the selected format and refreshes the view while preserving data consistency.

Outputs / Responses:

- Updated visual representation on screen and optional exportable report (PDF/CSV).

FReq 3: Notification and Alert System

Description:

- The system notifies managers about operational anomalies such as long queues, unexpected crowds, or irregular customer flow patterns.

Inputs / Stimuli:

- Analytic triggers from the customer activity module when metrics exceed predefined thresholds.

Process / Function:

- The rules engine evaluates threshold violations and generates alerts through the alert generator component.

Outputs / Responses:

- Real-time on-screen alerts with timestamp, event type, and priority level on the dashboard.

FReq 4: AI-Driven Insight Generation

Description:

- The system shall analyze behavioral and occupancy data to produce decision-support insights for management.

Inputs / Stimuli:

- Historical records from analytics modules (visitor counts, heatmaps, queue metrics).

Process / Function:

- The AI engine applies pattern recognition and trend analysis to detect repeating behaviors and generate textual recommendations.

Outputs / Responses:

- A list of insight statements displayed on the dashboard (e.g., "Consider adding staff in Zone A during peak hours").[7]

5.2.Use Case Diagram(s)

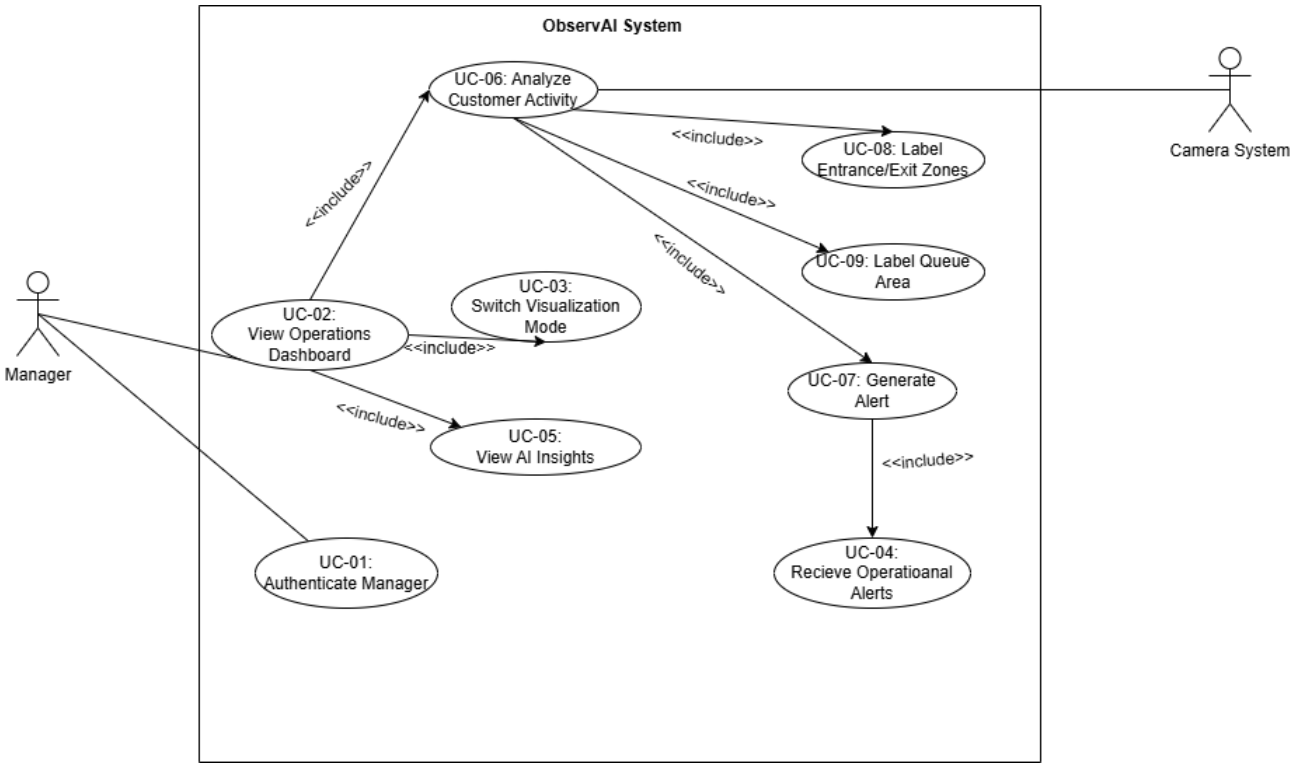

Figure 1 Use Case Diagram

[7] ChatGPT 5 Prompt: **"Write a part that qualifies these requirements by looking at our project's functional requirements"**

5.3.Mapping of Functional Requirements

Table 1 Cross Referencing Functional Requirements to Use Cases

Functional Requirement ID from Section 5.1	Use Case ID from Section 5.2	Initial Plan Functional Requirement ID [1]
FReq1: **Operations Dashboard** Displays AI-driven insights (visitor counting, demographics, queues, and heatmaps).	UC-01: Authenticate Manager UC-02: View Operations Dashboard UC-06: Analyze Customer Activity	FReq1: Operations Dashboard showing AI-driven customer activity & heatmaps
FReq2: Interactive Visualization Modes Allows managers to switch between bar, line, and heatmap views.	UC-03: Switch Visualization Mode UC-02: View Operations Dashboard	FReq2: Interactive Data Visualization
FReq3: Notification System Alerts managers about queues, crowd build-ups, or unusual traffic.	UC-04: Receive Operational Alerts UC-07: Generate Alert	FReq3: Real-Time Notifications and Alerts
FReq4: AI-Driven Insight Generation Analyzes behavioral and occupancy data to produce operational insights.	UC-05: View AI Insights UC-06: Analyze Customer Activity	FReq4: AI Insight Engine (Decision Support)

6. Non-Functional Requirements

6.1 Usability Requirements

ObservAI is designed for non-technical managerial users who rely on fast, intuitive interaction rather than technical configuration. The interface shall comply with modern web-usability standards and provide consistent navigation, visual clarity, and accessibility.

a. **Ease of Learning:**
   i. New users shall be able to complete fundamental operations—logging in, viewing live analytics, switching between reports, and exporting data—within 30 minutes of first use without external training.
   ii. A short built-in tutorial and labeled icons shall guide first-time users through the dashboard.[8]

b. **Consistency and Feedback:**
   i. Every screen shall conform to a common design template with standard coloring, spacing, and icon use.
   ii. The system shall enable instant visual feedback (like load indicators or status messages) for all user actions.

c. **Accessibility:**
   i. The contrast requirements for ObservAI will comply with WCAG 2.1 level AA guidelines.
   ii. Font sizes will remain readable when viewed in screens as small as 10 inches.

d. **Satisfaction Metrics:**
   i. In user evaluation tests, participants are expected to evaluate the interface as "easy to use" or "very easy to use."
   ii. There shall be fewer errors because of interface confusion.

[8] ChatGPT 5 Prompt: **"Write beginning of Usability Requirements in NFREQs and write first part which is Ease of Learning. Mention about short tutorials, easy to adapt etc."**

6.2 Performance Requirements

Aspects related to performance directly influence decision-making in cafes. As a result, ObservAI must run in real-time.

a. **Response Time:** The result of live camera analytics (object detection & heatmap updation) shall be visible on the dashboard within 2 seconds after capturing data.
b. **Concurrent Streams:** The system shall be able to accommodate a maximum of 2 or 3 live video streams per site.
c. **Throughput:** The backend shall achieve a minimum of 10 frames per second (FPS) per active video stream.
d. **Data Queries:** Historical reports shall load within 3 seconds.
e. **Backup and Recovery:** Automatic database backups shall occur every 24 hours; full data restoration shall complete within 15 minutes after failure.
f. **AI Accuracy:** Computer-vision models shall maintain suitable precision and recall metrics in crowd counting and demographic estimation.

6.3 Software System Attributes

a. Reliability

i. ObservAI shall operate continuously uptime during business hours (08:00–00:00).
ii. Transaction logs shall record every analytic operation to prevent data loss after unexpected shutdowns.

b. Availability

i. Scheduled maintenance downtime shall not exceed 2 hours per month.
ii. In case of network interruption, edge caching shall store the last 30 minutes of analytics locally and synchronize automatically once the connection is re-established.

c. Security

i. All communications between client, server, and database shall employ TLS 1.3 encryption.
ii. The system shall sanitize and anonymize every video frame immediately after AI inference; no raw visual data shall be permanently stored.

iii. Data handling shall comply with KVKK and GDPR principles: purpose limitation, data minimization, and right to erasure.[9]

d. Maintainability

i. ObservAI shall employ a modular, layered architecture separating:

- Presentation Layer (React frontend)
- Logic Layer (Node.js controllers)
- Data Layer (MongoDB or SQL database)
- Each component shall be updatable independently.
- Version control shall be managed through GitHub with branch-based reviews.

e. Portability

i. The platform shall function identically across the latest two versions of Chrome, Edge, Safari, and Firefox.

ii. It shall be deployable on Windows and macOS servers without code modification.

iii. All services shall run inside Docker containers to ensure consistent behavior across environments.

Migration to cloud platforms such as AWS ECS or Azure Container Apps shall require minimal configuration.

6.4 Constraints

a. ObservAI shall not include functionalities such as facial recognition, voice analytics, or direct payment handling.

b. Data processing shall occur exclusively within legally compliant regions in accordance with Turkish Law No. 6698 (KVKK).

c. The system shall remain fully functional on local café networks.

6.5 Error-Handling Requirements

a. **AI Detection Failures:** If a camera feed stops or frames become unreadable, the inference service shall retry within 5 seconds and log the incident.

[9] ChatGPT 5 Prompt: **"Can you add additional Software System Attributes related to Security of System such HTTPS/TLS usage and KVKK limitations. "**

b. **Database Errors:** Failed writes shall queue for re-attempts; duplicates shall be avoided via primary-key enforcement.
c. **Critical Exceptions:** Unrecoverable backend errors shall generate structured logs for the admin.
d. **User Errors**: The interface shall display non-technical, plain-language messages rather than stack traces.

6.6 Other Non-Functional Requirements

a. **Localization:** Interface language shall switch between English and Turkish; numeric and date formats shall adapt to locale.
b. **Scalability**: System architecture shall allow integration of additional analytics modules (e.g., queue-time prediction) without code refactoring
c. **Auditability**: Each analytic event shall be traceable from input to visualization for transparency and debugging.

7. Logical Database Requirements

In a structured database environment, ObservAI keeps track of user credentials, camera settings, analytics information, alert logs, and insights produced by artificial intelligence. The system makes use of SQLite for short-term edge caching and PostgreSQL with the TimescaleDB extension for long-term time-series data. To guarantee consistency and effective data retrieval, relationships between users, sites, cameras, analytics records, alerts, and insights are established, and each table has well-defined attributes.

The database handles several categories of information:

- **User Accounts**: Contain login credentials, email verification data, and access permissions for café and restaurant managers.
- **Camera and Site Configurations:** Define café locations, associated cameras, and alert thresholds for each area.
- **Analytics and Heatmaps:** Store real-time visitor counts, occupancy metrics, queue lengths, and spatial density maps generated by the AI engine.
- **Alerts and Notifications:** Record unusual activity or crowding events detected by the system.
- **AI Insights:** Contain natural-language summaries and recommendations produced by the integrated Gemini LLM model.

Different data types are accessed at varying frequencies.

Analytics and heatmap data are written frequently (every few seconds) by the AI engine, while configuration and user data are modified less often. Dashboard queries for visualizations occur periodically as managers interact with the interface.

Access control is managed through a role-based model.

Managers can view analytics, alerts, and insights for their own cafés, while system administrators have broader privileges for configuration and maintenance. Direct access to the database is limited to backend services and authorized developers.

Key entities include User, Site, Camera, AnalyticsRecord, Alert, and Insight.

Each analytics record references a valid camera and site to maintain data integrity. Constraints ensure that data such as visitor counts and queue lengths are non-negative, and time-stamped analytics remain unique for each camera. Alerts and insights are linked to corresponding analytics records for traceability.

Data integrity is maintained through primary and foreign keys, validation rules, and referential constraints. Deletion of a site automatically cascades to related cameras and analytics only when explicitly approved by an administrator. Audit logs record all major data modifications.

Data retention policies balance operational utility and privacy.

Short-term analytics and heatmap data are retained for up to seven days to support recent trend visualization. Aggregated summaries and AI insights are kept for up to three months for managerial analysis. User accounts and configuration data are preserved as long as the system remains active, unless deleted by an authorized administrator. All stored information complies with KVKK and GDPR data protection principles, ensuring anonymization, limited retention, and privacy by design.[10]

[10] ChatGPT 5 Prompt: **"Write this section according to this opening and those requirements for our project."**

8. Verification

This section outlines how the ObservAI software will be verified to ensure that it fully satisfies all of the functional and non-functional requirements mentioned in the previous sections. Verification activities include simulation-based validation, continuous integration checks, and a combination of automated and manual testing. These techniques are meant to verify that ObservAI's AI-powered analytics, visualization, and alert systems function precisely, effectively, and safely in actual restaurant and café settings.

8.1.Verification Strategy

ObservAI follows an incremental verification approach aligned with the agile principles. To ensure that implemented features meet their respective functional and non-functional requirements, requirement-based testing is conducted at the end of each increment.

Verification will proceed in the following order:

- **Backend verification:** Node.js modules, API endpoints, and AI integration layers will be validated first.
- **Frontend verification:** React-based dashboard components will be tested manually through browsers to ensure responsiveness and correct visualization.
- **System-level verification:** Integrated testing will confirm that all system components communicate correctly through secure RESTful APIs.
- **Continuous Integration (CI):** A lightweight CI pipeline (via GitHub Actions or equivalent) will run build tests and available backend unit tests on each commit to prevent regressions.

Most testing will be manual, supported by selected automated checks for API endpoints, system responses, and functional validation of key modules.

8.2.Functional Requirements Verification

FReq1: Operations Dashboard

Goal: Validate the user is authenticated and correct display and real-time updating of customer analytics (visitor counts, demographics, heatmaps).

Method:

- Use simulated RTSP video streams or pre-recorded videos as test inputs.

- Compare displayed analytics (e.g., visitor counts) with manually counted results (ground truth).
- Verify that real-time heatmaps correspond to actual movement zones.

Tools: Node.js scripts, Postman, Chrome DevTools.

FReq2: Interactive Visualization Modes

Goal: Ensure visualization switching (bar, line, heatmap) functions without data loss or inconsistencies.

Method:

- Manually switch between visualization modes while observing data persistence.
- Confirm numerical consistency across chart types.

Tools: Manual browser testing (Chrome, Edge, Firefox).

FReq3: Notification System

Goal: Confirm that alerts are correctly generated when abnormal crowd levels or queues are detected.

Method:

- Simulate threshold violations using synthetic data.
- Verify that alerts appear in the dashboard and are delivered via Web Push / FCM.

Tools: Node.js backend simulations, Postman, browser notification test environment.

FReq4: AI-Driven Insight Generation

Goal: Verify that the LLM-generated insights (Gemini 2.0 Flash) are relevant, accurate, and easily understandable.

Method:

- Use predefined analytics scenarios (e.g., peak hours, low occupancy zones).
- Compare generated recommendations against expected outcomes.
- Conduct a brief expert or managerial evaluation of insight clarity.

Tools: Node.js test harness, LLM API integration logs.[11]

[11] ChatGPT 5 Prompt: **"Write this section according to this opening and those requirements for our project. And also look at our past chats about ObservAI while answering"**

8.3.Non-Functional Requirements Verification

NFReq1: Web Accessibility on Desktop and Mobile

Approach:

Test ObservAI's web interface on multiple browsers (Chrome, Edge, Safari, Firefox) and across different screen resolutions (desktop, tablet, mobile).

Check responsiveness, layout stability, and usability under varying display sizes.

Manual browser tests will verify that all dashboard components, analytics visuals, and buttons remain accessible and functional.

NFReq2: Daily Database Backups and Recovery

Approach:

Simulate data loss and confirm that automated daily backups restore the database correctly.

Test the backup and recovery scripts by intentionally deleting a small dataset segment, restoring it, and comparing integrity with the original copy.

Observe log entries to verify that backup events are triggered every 24 hours without failure.

NFReq3: Encrypted Communication (HTTPS/TLS)

Approach:

Verify HTTPS/TLS 1.3 configuration between frontend, backend, and database servers using browser security indicators and SSL validation tools.

Conduct basic penetration testing with OWASP ZAP to ensure no unencrypted data packets or open HTTP endpoints exist.

Inspect API traffic using Postman to confirm that all data exchange occurs exclusively over secure HTTPS connections.

NFReq4: Data Privacy and Anonymization (KVKK & GDPR Compliance)

Approach:

Inspect AI-processed frames to verify that anonymization is applied immediately after inference (no raw visual data stored).

Confirm that metadata logs do not include personally identifiable information (PII).

Review anonymization scripts, deletion policies, and database schemas for compliance with KVKK and GDPR standards (purpose limitation, data minimization, right to erasure).[12]

8.4.Tools and Continuous Verification

To ensure continuous compliance with requirements, the following tools and techniques will be employed:

Table 2 Test Tools & Purposes[13]

Verification Area	Tool / Method	Purpose
Backend Testing	Node.js scripts, Postman	Validate API functionality and data flow
Frontend Testing	Manual browser tests	Confirm UI responsiveness and visualization accuracy
AI & Insight Validation	Gemini API logs, scenario datasets	Ensure relevant and understandable AI outputs
Performance Monitoring	Browser DevTools, backend logs	Measure response time and FPS
Security Verification	OWASP ZAP, HTTPS/TLS configuration	Confirm encryption and access control
CI & Automation	GitHub Actions	Run build tests and regression checks automatically

[12] ChatGPT 5 Prompt: **"Write this section to this non functional requirements for our project. Check the requirements while answering"**

[13] ChatGPT 5 Prompt: **"Draw a table for us to show which technologies we will use according to the added verification part."**

9. Discussions

In this part, we describe the issues encountered by our team in preparing its "ObservAI" project's System Requirements Specification document. In preparing the document, members undertook a series of analyses to achieve coordination in requirements. These problems encountered are classified into:

1. Limitations and Constraints

In preparing our SRS, some limitations are faced for our scholarly productivity:

- **Time Constraints:** The preparation for the SRS was happening simultaneously with mid-term exams for various courses. It was difficult to manage both academic responsibilities and paperwork for projects. Time management helped to address both requirements.
- **Limited Documentation Experience:** Although all members have a technical background, preparing a detailed and IEEE-compliant SRS document required understanding new standards and formal documentation methods, which initially slowed down progress.[14]
- **Tool Limitations:** The team relied primarily on freely available software tools such as Google Docs, Draw.io, and GitHub for collaborative editing and diagram creation. While effective, these tools had certain limitations in advanced formatting and UML diagram standardization.[15]

2. Health and Safety Issues

[14] ChatGPT 5 Prompt: **"Can you add documentation experience skills which slow down our progress while writing the document. Write clear, formal and -documentation difficulty part-."**
[15] ChatGPT 5 Prompt: **"Can you add Tool Limitation part. It should be related to the deficit of free source tools."**

No direct health or safety issues were encountered during the preparation of this document. However, extended screen time and late-night online meetings during the midterm period contributed to fatigue and reduced concentration in some team sessions.[16]

3. Legal Issues

There are no legal issues related to the preparation for SRS. But since ObservAI uses camera analytics and image processing analysis, future compliance with data protection legislation like KVKK or GDPR will become important later.

4. Economic Issues and Constraints

As a student-led project without external funding, we operated under financial limitations:

- **No Dedicated Budget:** Lack of budget makes it harder to use commercial AI or UML design tools.

- **Resource Dependence:** We exclusively relied on open-source resources that are freely available to use for our AI modeling. Although we conformed to sustainability principles in our project, there was a limitation in functionality.

5. Sustainability

During SRS development, sustainability was not emphasized. The major aim was to define requirements. In subsequent developments of ObservAI, sustainability will nonetheless be ensured since there will be optimization in computational resources used in camera analytics.

6. Ethical Issues

There are no ethical considerations that came up in the entire documentation phase. The development team was aware of ethical issues related to using AI. These are related to anonymization of data processing in ObservAI. The solution does not involve face recognition to meet privacy-related ethical requirements.

[16] ChatGPT 5 Prompt: **"Can you write a part related to health issues. You can mention about long screen times which can harm our health. Write clearly and shortly. "**

7. Multidisciplinary Collaboration

All team members are students within the **CTIS Department**, sharing similar technical backgrounds. While this ensured alignment in terminology and communication, it limited interdisciplinary perspectives (e.g., design, psychology, or business management). Despite this, internal collaboration was strong, and roles were distributed efficiently among documentation, understanding AI model, and architectural design tasks.[17]

8. Additional General Challenges

In addition to these specific limitations, there are some general issues that are encountered:

- **Communication and Coordination:** Scheduling online team sessions was sometimes made challenging because team members' schedules conflicted due to courses or exam schedules. Online resources like Discord, Notion, Zoom and Whatsapp helped in asynchronous coordination.
- **Requirement Elicitation:** ObservAI's users (café managers) are hypothetical. Requirements are derived using assumptions made, prior research, or interviewing local café owners. There was a slight uncertainty involved in some user requirements concerning ObservAI's user interface.
- **Technical Depth:** Certain requirements like processing limits related to performance or AI inference speed limits demanded additional research to achieve realistic values. These values had to be verified to correctly represent computer vision processing requirements.
- **Formatting and Standardization:** Aligning the document format with the official IEEE SRS template required additional review cycles, particularly for section numbering, heading consistency, and terminology alignment.[18]

[17] ChatGPT 5 Prompt: **"Can you add the Multi-Colloboration part for the following part mentioning as a group we lack working together, interdisciplinary working. Also give solution how can we handle with AI"**

[18] ChatGPT 5 Prompt: **"Can you add Formatting Challenges for Additional Part at the end. It should related to IEEE template such as terminology and layout"**

10. References

[1] F.E.Partal, H.İ.Atalay, M.Yılmaz, S.Şengül, and U.A.Karakış, "Initial Plan of Team 12," Bilkent University, Department of Computer Technology and Information Systems, Ankara, Türkiye, October 2025.